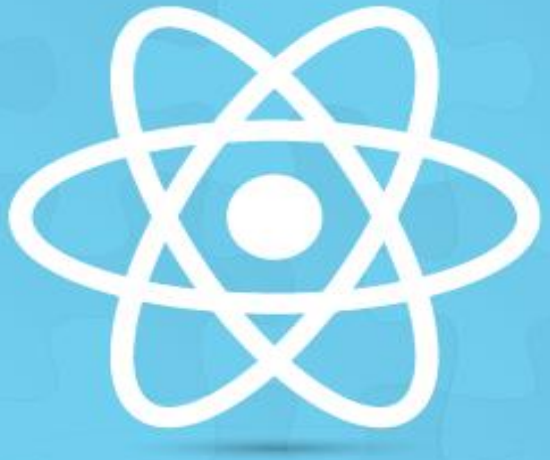
Intitulé du cours:
Technique des applications web

Chapitre 3 :
Les composants React

Enseignante: Abid Arij

Année de formation 2025-2026

React JS



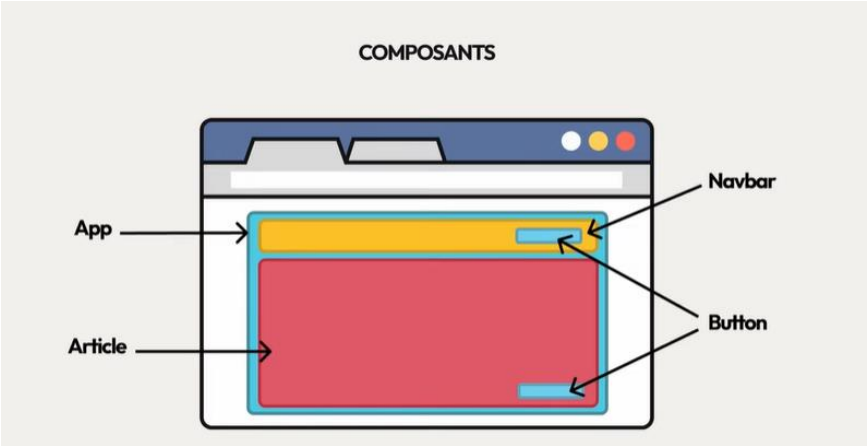
Les composants React

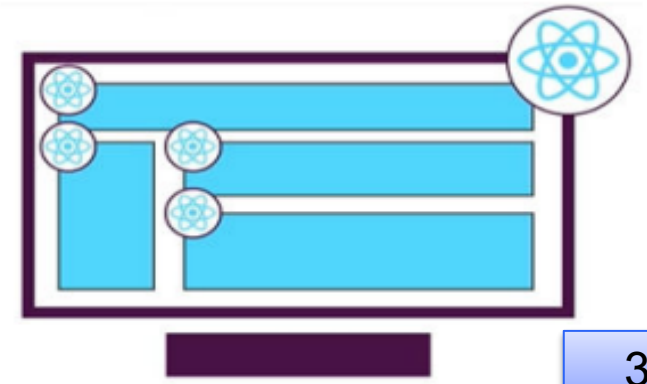
C’est quoi un composant dans React ?

- Un **composant** est un **bloc de code réutilisable** qui représente une partie de l’interface utilisateur.

=> On peut le comparer à une pièce LEGO :

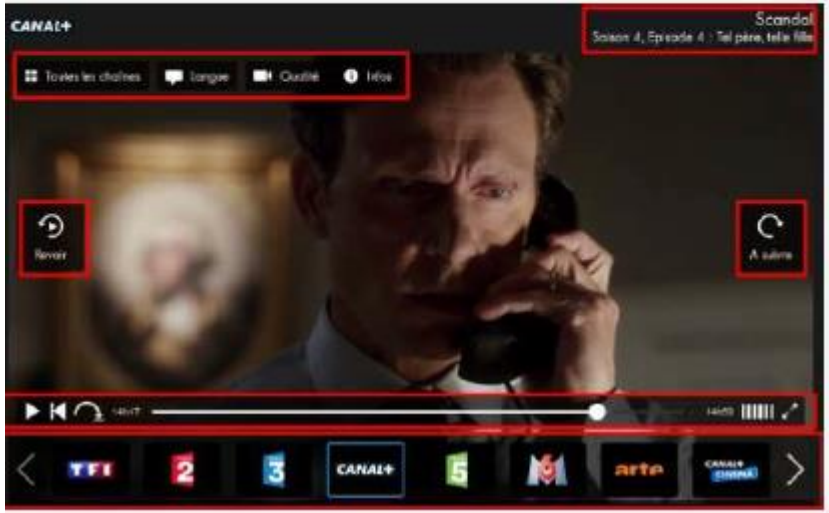
- Chaque pièce = un composant
- Plusieurs pièces assemblées = une application complète





C’est quoi un composant dans React ?

- **React :** Arbre de composants
  - Le tout n’est que composant !
    - Et un composant est composable.

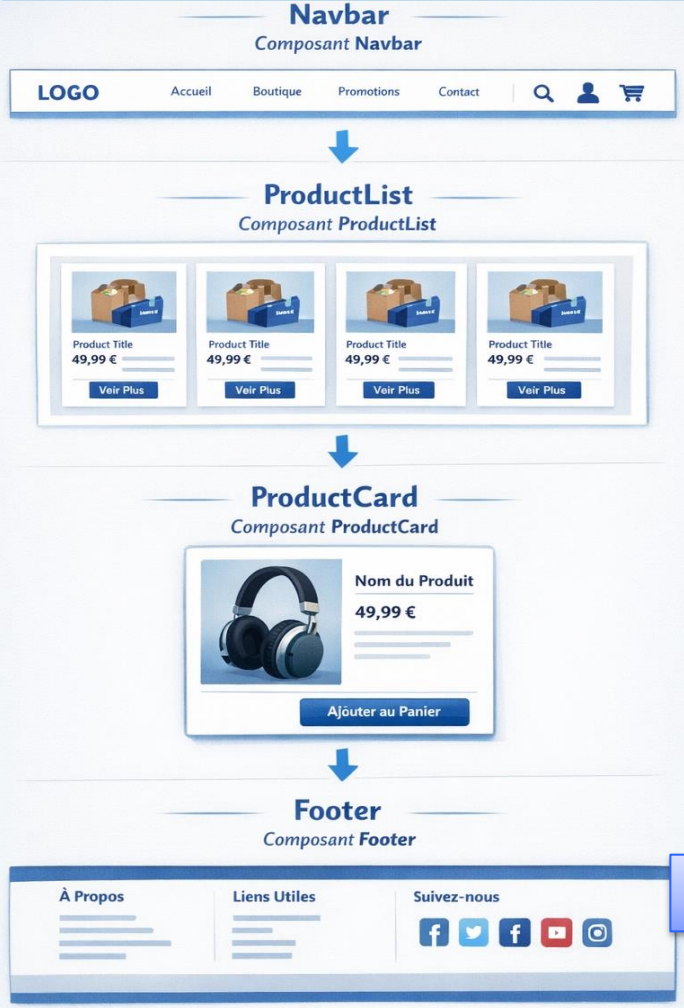


C’est quoi un composant dans React ?

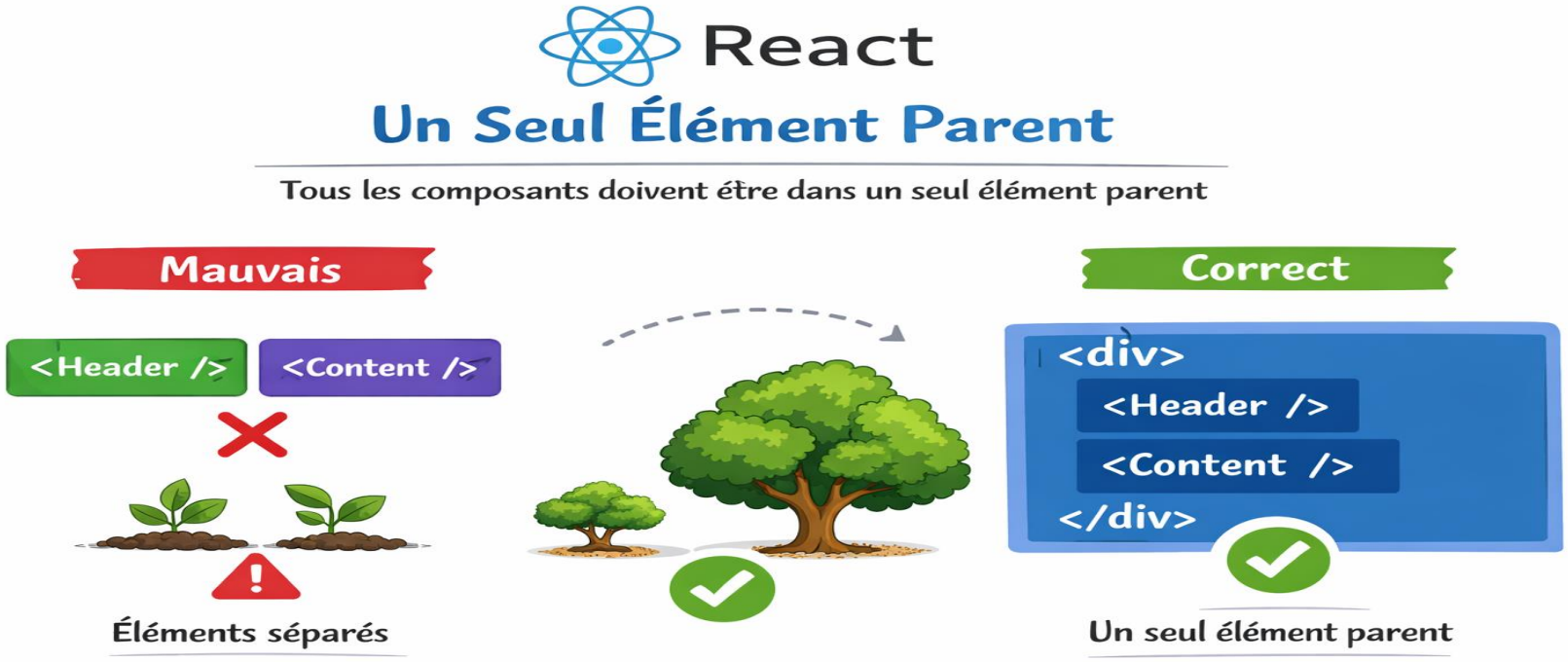
Dans une application **e-commerce :**

- Barre de navigation → composant Navbar
- Liste des produits → composant ProductList
- Carte produit → composant ProductCard
- Pied de page → composant Footer

- Chaque élément
  est un **composant indépendant**.



“La Règle du Parent Unique en React”



⚠️ Les règles importantes

✅ Nom en majuscule

Un composant commence TOUJOURS par une majuscule : `MonComposant`, jamais `monComposant`

✅ Retourne du JSX

Le composant doit toujours retourner quelque chose à afficher : du HTML, d'autres composants, ou `null`

✅ Un seul parent

Le return doit avoir un seul élément parent. Utilise `<>` `</>` si besoin.

✅ Réutilisable

Un composant peut être utilisé autant de fois que tu veux dans ton app !

Pourquoi utiliser des composants ?

- **Réutilisation**

On écrit une fois, on utilise plusieurs fois.

- **Organisation**

Code clair et structuré.

- **Maintenance facile**

Modifier un composant = changer partout.

- **Travail en équipe**

Chaque développeur peut travailler sur un composant.

Les types de composants

Il existe deux grands types :

1) Composant Stateless (sans état)

Appelé aussi :

- Composant présentatif
- Composant simple

Il affiche seulement les données.

- Il ne gère pas :
- des variables dynamiques
- des changements

Les composants React

- **Composants React :**
  - Ils servent le même objectif que les fonctions JavaScript, mais fonctionnent de manière isolée et renvoie du HTML via une fonction de rendu.
  - **render() :** fonction de base de React pour afficher du contenu à l'écran,
  - Ils peuvent être définis sous forme d'une fonction ou d'une classe
  - Ils sont indépendants et réutilisables.

Composant purement fonctionnel (stateless)

- Composant sous forme d'une fonction javaScript

```
function App() {
  return (
    <h2>Bienvenue à cette formation React</h2>
  );
}
export default App;
```

Composant de classe (statefull) généralement contenant un état

```
class App extends React.Component {
  render(){
    return (
      <h2>Bienvenue à cette formation React</h2>
    );
  }
}
export default App;
```

Composant de classe sous forme de classe et hérite de la classe React.Component

Les composants React : Notion du state

- C'est quoi le « state » en React ?
  - objet, où l'on va pouvoir stocker les données relatives à un composant donné
  - accessible uniquement à travers son composant
  - va être utilisable uniquement dans les composants "stateful"
- Création du state :
  - **this.state** = { définition *des données sous forme de (key : value)* }

```
// On va créer notre state dans la méthode constructor() de notre composant
constructor(props) {
  super(props); // Nous permet d'utiliser les méthode de React.Component
  this.state = {
    // On stocke ici notre data
    monTexte: "Bienvenue à cette formation React."
  };
}
```

Les composants React : Notion

- Utilisation du « state » :
  - L'accès aux données stockées dans le state se fait à travers : ***this.state.nomAttribut***
  - Pour l'affichage, on utilise la notation { } (on parle d'interpolation)

```
class App extends Component {
  // On va créer notre state dans la méthode constructor() de notre composant
  constructor(props) {
    super(props); // Nous permet d'utiliser les méthode de React.Component
    this.state = {
      // On stocke ici notre data
      monTexte: "Bienvenue à cette formation React."
    };
  }
  render() {
    return (<h2>{this.state.monTexte}</h2>);
  }
}
export default App;
```

Les composants React : Notion du state

- Modification du « state » :
  - Se fait via la méthode ***this.setState*** à laquelle on passera un objet avec l'attribut à modifier et la nouvelle valeur à appliquer.

```
this.state={
    monTexte: ''
}
```

```
this.setState(
    {
        monTexte:"Bonjour à toutes et à tous !"
    });
```

Les composants React : Notion

- Modification du « state » :
  - Définition de l'action au click :
    - Pour cet exemple : on met à jour l'état en incrémentant la valeur courante du compteur
    - React effectue un rendu intégral d'un composant à chaque changement des données qui lui sont associées,

```
handleClick = () => {
  this.setState({
    value : this.state.value + 1
  })
}
```

```
<input type="button" onClick={()=>this.handleClick()} value="Incrémenter"/>

<h2>{this.state.value}</h2>
```

Les composants React : Listes et clés

- Notion de map :
  - méthode afin de parcourir d'une manière simple les éléments d'un tableau (Array)
  - Transformer un tableau d'objets en un tableau d'éléments JSX

❑ Exemple 1 :

```
const numbers = [1, 2, 3, 4, 5];
const listItems = numbers.map( (number) => number )
```

- Sous forme d'une liste :

```
const listItems = numbers.map((number) =>
  <li>{number}</li>
);
```

```
▸Warning: Each child    react-jsx-dev-runtime.development.js:87
in a list should have a unique "key" prop.

Check the render method of `App`. See https://reactjs.org/link/w
arning-keys for more information.
    at li
    at App
```

- **Solution :**

```
const listItems = numbers.map((number, ind) =>
  <li key={ind}>{number}</li>
);
```

Utiliser une clé unique pour identifier chaque élément.

Les composants React : Listes et clés

❑ Exemple 2 :

```
state = {
        monTexte: 'Bienvenue à notre formation React !',
        personnes: [  { id: 1, nom: 'Mohamed' },
                      { id: 2, nom: 'Salah' },
                      { id: 3, nom: 'Ali' } ]
    }
```

```
render() {
        return (<div>
                {
                    this.state.personnes.map((p, i) =>
                        <li key={i}>{p.id} -- {p.nom}</li>)
                }
                </div>
        );
    }
```

Les types de composants

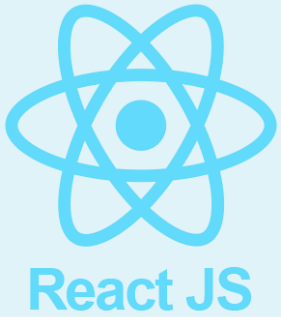
Composant purement fonctionnel (stateless)

```
function App() {
  return (
    <h2>Bienvenue à cette formation React</h2>
  );
}
export default App;
```

Composant de classe (statefull) généralement contenant un état

```
class App extends React.Component {
  render(){
    return (
      <h2>Bienvenue à cette formation React</h2>
    );
  }
}
export default App;
```

Cycle de vie d'un composant React



Cycle de vie d'un composant React

- Le **cycle de vie d'un composant React** décrit les différentes étapes par lesquelles passe un composant, depuis sa création jusqu'à sa suppression du DOM.
C'est un concept central pour comprendre **quand exécuter du code (API, timers, effets, nettoyage, etc.)**.

Les grandes phases du cycle de vie

On peut découper le cycle de vie en **3 phases principales** :

1. **Montage (Mounting)**
   Le composant est **créé et inséré dans le DOM**
2. **Mise à jour (Updating)**
   Le composant est **re-rendu suite à un changement (state ou props)**
3. **Démontage (Unmounting)**
   Le composant est **supprimé du DOM**

Cycle de vie avec les composants

Flux d'exécution des phases

🌱 PHASE 1 — MONTAGE

Constructeur / init → render() / JSX → DOM réel mis à jour → useLayoutEffect → useEffect (montage)

🔄 PHASE 2 — MISE À JOUR

Props / State changent → Re-render JSX → Reconciliation (diff) → Cleanup useEffect → useEffect (re-run)

🗑 PHASE 3 — DÉMONTAGE

Composant retiré → Cleanup useEffect → Cleanup useLayoutEffect → Nœud DOM supprimé

Merci pour

votre Attetion